Hi [Name],

I'm ready to move forward with the build for "Terminal Buddy."

After an extensive planning phase, we have locked the design and architecture to prioritize system responsiveness, cognitive accessibility, and long-term memory logging. I'm attaching the Master Design Spec (`TERMINAL_BUDDY_MASTER_SPEC.md`), which covers the entire project vision, but I need you to focus strictly on **Phase 1: The Iron Skeleton** for this initial implementation.

Below are the clear, professional instructions, dependency locks, and starter templates for our first module.

1. Architectural Refresher: The Sidecar & Memory Integration

To ensure the 120FPS GUI never drops a frame, we are strictly enforcing a **Sidecar Pattern**.

- **Process A (`buddy-gui`):** Pure rendering (Rust + GPUI). It only captures keystrokes and draws grids.
- **Process B (`buddy-daemon`):** The heavy lifter. It runs the pseudo-terminal (PTY), handles AI processes, and—crucially—manages the **Memory Integration**.

The Memory Hook: As the Daemon receives stdout from the PTY, it must both stream that text to the GUI via IPC *and* prepare to write it to our Memvid Write-Ahead Log (`.mv2` format). Phase 1 only requires the PTY streaming, but the architecture must leave the hook open for the Memvid integration in Phase 3.

2. Workspace Setup & Dependency Lock (`Cargo.toml`)

Please initialize a Cargo workspace to keep the GUI and Daemon cleanly separated.

Directory Structure:

```
terminal-buddy/
├── Cargo.toml
├── shared/       # IPC message definitions (Bincode)
├── gui/          # GPUI frontend
└── daemon/       # Tokio + PTY backend
```

Root `Cargo.toml` Requirements:

```
[workspace]
members = ["gui", "daemon", "shared"]
resolver = "2"

[workspace.dependencies]
# GUI / Rendering
gpui = { git = "[https://github.com/zed-industries/zed](https://github.com/zed-ir
alacritty_terminal = "0.22"

# Daemon / System
tokio = { version = "1.40", features = ["full"] }
portable-pty = "0.8"
sysinfo = "0.30"

# IPC & Memory Data
interprocess = "2.2"
bincode = "1.3"
serde = { version = "1.0", features = ["derive"] }
```

3. Module 1 Starter Template: The Daemon PTY

To ensure we get the Sidecar logic right from day one, here is the architectural stub for the Daemon's PTY loop. This demonstrates where the GUI IPC and the Memory Logging will intersect.

`daemon/src/pty_runner.rs` **(Architectural Stub):**

```
use portable_pty::{CommandBuilder, NativePtySystem, PtySize, PtySystem};
use std::sync::mpsc::Sender;
use tokio::io::{AsyncReadExt, AsyncWriteExt};

pub async fn spawn_sidecar_pty(ipc_sender: Sender<Vec<u8>>) -> anyhow::Result<()>
    let pty_system = NativePtySystem::default();
    let pair = pty_system.openpty(PtySize {
        rows: 24,
        cols: 80,
        pixel_width: 0,
        pixel_height: 0,
    })?;

    let mut cmd = CommandBuilder::new(if cfg!(windows) { "cmd.exe" } else { "bash
    let _child = pair.slave.spawn_command(cmd)?;

    let mut reader = pair.master.try_clone_reader()?;

    // The Core Sidecar Loop
```

```
    tokio::task::spawn_blocking(move || {
        let mut buf = [0u8; 1024];
        while let Ok(n) = reader.read(&mut buf) {
            if n == 0 { break; }
            let output = buf[..n].to_vec();

            // 1. Send to Process A (GUI) for instant rendering
            let _ = ipc_sender.send(output.clone());

            // 2. TODO (Phase 3 Memory Integration):
            // Send `output` to Memvid logger along with sysinfo Delta here.
            // memvid_writer.append_frame(output, current_sys_delta);
        }
    });

    Ok(())
}
```

4. Repository Standards & Professional Guidelines

To maintain a professional codebase for Terminal Buddy, please adhere to the following strict standards from the first commit:

1. **Error Handling:** Do not use `.unwrap()` or `.expect()` in production paths. Use `anyhow` for the application layer and `thiserror` for library layers. Return graceful errors to the GUI so the "Buddy" can explain crashes to the user.
2. **Formatting & Linting:** Ensure `cargo fmt` and `cargo clippy` pass on every commit. Add `#![deny(clippy::unwrap_used)]` to your `lib.rs` and `main.rs` files.
3. **Asynchronous Boundaries:** Never execute heavy blocking I/O (like writing to the `.mv2` memory file) on the main Tokio runtime thread, and *never* block the GPUI render thread. Use `tokio::task::spawn_blocking` for system-level reads.

The primary goal of Phase 1 is a stable, non-blocking window where I can type and see the shell output. Let me know once you have the workspace structured and the initial GPUI window rendering.

Thanks, [Your Name]